

```
#!/usr/bin/env python3
"""
OTR News daily article writer.
Asks Claude (via the Anthropic API, with web search) to research the most importa
trucking story for owner-operators and small fleets and write an original article
The article is saved to posts/ as a DRAFT. To publish it, edit the file on GitHub
and change `draft: true` to `draft: false`.
Needs the ANTHROPIC_API_KEY secret. Standard library only.
"""

import json
import os
import re
import sys
import urllib.error
import urllib.request
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).parent
POSTS = ROOT / "posts"
ARCHIVE = ROOT / "data" / "items.json"
GUIDE = ROOT / "style-guide.md" # optional: your own voice and rules
MODEL = os.environ.get("OTR_MODEL", "claude-sonnet-5-5")
MAX_SEARCHES = 8

PROMPT = """You are the staff writer for OTR News (otrnews.com), a trucking news
owner-operators, small fleets (1-10 trucks), and company drivers. Today is {today}

Your job: find the single most useful trucking story from the last 48 hours for t
audience, research it with web search, and write an ORIGINAL article about it.

Pick stories that affect drivers' and small carriers' money, time, or compliance:
FMCSA/DOT rules and enforcement, registration and authority, freight rates and de
diesel prices, broker issues, fraud and cargo theft, insurance, parking, equipmen
driver pay. Skip stories only large carriers or investors care about.

Recent OTR News articles (do not repeat these topics unless there is major new ne
{recent}

Today's industry headlines, as leads (verify everything yourself; do not rely on
{headlines}

Rules:
- Write every sentence in your own words. Never copy or closely paraphrase anothe
```

- outlet's sentences or structure. Report the facts, then explain what they mean.
- Only state facts you found in your research. If something is unclear or unconfirmed, say so. Never invent quotes, numbers, names, or dates.
- Prefer primary sources (FMCSA, DOT, Federal Register, courts, company statement)
- Plain, direct language a driver would use. No hype, no filler.
- Write as a newsroom, in the third person. Never use "I" or "we", never address editor, and never mention your research process, searches, or what you could not do. If a detail is unconfirmed, say so in reader-facing terms, for example "FMCSA had not published the notice as of Tuesday."
- The headline must not contain a colon. Use plain words instead.
- Structure: a short opening that says what happened and why it matters; sections "## " headings; a "## What to do" section with practical steps when relevant; a \*\*Bottom line:\*\*; then "## Sources" with 2-5 markdown links you actually used.
- 450-800 words.

{guide}

Output ONLY the finished file, starting with the front matter, exactly in this format:

---

title: <headline, specific and plain, under 90 characters>

date: {iso}

summary: <one or two sentences, under 200 characters>

category: <one of: Regulations, Freight market, Fuel, Safety, Equipment, Drivers,

author: OTR News Staff

draft: true

---

<article body in markdown>

"""

```
def recent_posts(n=10):
```

```
    titles = []
```

```
    for f in sorted(POSTS.glob("*.md"), reverse=True)[:n]:
```

```
        m = re.search(r"^title:\s*(.+)$", f.read_text(encoding="utf-8"), re.M)
```

```
        if m:
```

```
            titles.append("- " + m.group(1).strip())
```

```
    return "\n".join(titles) or "- (none yet)"
```

```
def headlines(n=25):
```

```
    try:
```

```
        items = json.loads(ARCHIVE.read_text())
```

```
    except Exception:
```

```
        return "- (none available)"
```

```
    return "\n".join(f"- {i['title']} ({i['source']})" for i in items[:n] if not
```

```

def call_claude(messages, key):
    body = json.dumps({
        "model": MODEL,
        "max_tokens": 4000,
        "messages": messages,
        "tools": [{"type": "web_search_20250305", "name": "web_search", "max_uses
    }).encode()
    req = urllib.request.Request("https://api.anthropic.com/v1/messages", data=bo
        "x-api-key": key, "anthropic-version": "2023-06-01", "content-type": "app
    try:
        with urllib.request.urlopen(req, timeout=600) as r:
            return json.loads(r.read())
    except urllib.error.HTTPError as e:
        sys.exit(f"Anthropic API error {e.code}: {e.read().decode()[:500]}")

def main():
    key = os.environ.get("ANTHROPIC_API_KEY", "").strip()
    if not key:
        print("No ANTHROPIC_API_KEY secret set; skipping the daily article.")
        return

    now = datetime.now(timezone.utc)
    guide = ""
    if GUIDE.exists():
        guide = "\nHouse style from the publisher (follow it):\n" + GUIDE.read_te
    prompt = PROMPT.format(today=now.strftime("%A, %B %-d, %Y"), iso=now.strftime
        recent=recent_posts(), headlines=headlines(), guide=gu
    messages = [{"role": "user", "content": prompt}]

    for _ in range(5): # server-side search can pause a long turn; continue it
        resp = call_claude(messages, key)
        if resp.get("stop_reason") != "pause_turn":
            break
        messages += [{"role": "assistant", "content": resp["content"]}

    text = "".join(b.get("text", "") for b in resp.get("content", [])) if b.get("t
    start = text.find("---")
    if start == -1 or "title:" not in text:
        sys.exit("The reply didn't contain an article. Raw reply:\n" + text[:2000
    article = text[start:].strip() + "\n"

    # Always save as a draft, whatever the model wrote
    head_end = article.find("---", 3)
    head = article[:head_end]
    head = re.sub(r"^draft:.*$", "draft: true", head, flags=re.M)
    if "draft:" not in head:

```

```

        head = head.rstrip() + "\ndraft: true\n"
    article = head + article[head_end:]

# House-style cleanup: no colons in the headline/summary, no notes to the edi
def no_colon(m):
    return m.group(1) + m.group(2).replace(": ", " - ")
article = re.sub(r"^(title:\s*|summary:\s*)(.+)$", no_colon, article, flags=r
head_end = article.find("---", 3)
body = article[head_end + 3:]
paras = re.split(r"\n\s*\n", body)
note = re.compile(r"^(I|We)\b|\b(I (did|could|was|found|searched|couldn't|did
kept = [p for p in paras if not note.search(p.strip())]
article = article[:head_end + 3] + "\n\n".join(kept)

title = re.search(r"^title:\s*(.+)$", article, re.M).group(1).strip().strip('
slug = re.sub(r"^[a-z0-9]+", "-", title.lower()).strip("-")[:60].rstrip("-")
POSTS.mkdir(exist_ok=True)
path = POSTS / f"{now:%Y-%m-%d}-{slug}.md"
path.write_text(article, encoding="utf-8")
print(f"Draft saved: {path.relative_to(ROOT)}")

if __name__ == "__main__":
    main()

```